

Niveles de Testing de Software

De la unidad más pequeña de código hasta la aceptación del usuario: un mapa completo para entender dónde y cómo se prueba el software.

¿Qué vamos a ver hoy?



1. Pruebas Unitarias

Aislar y verificar la pieza más pequeña de código.



2. Pruebas de Integración

Verificar que los módulos y las APIs se hablen bien entre sí.



3. Sistema y Regresión

Validar el flujo end-to-end y que nada se haya roto.



4. UAT y Exploratorias

Confirmar que el software realmente sirve al usuario.

La pieza más pequeña del rompecabezas

- 🔍 **Qué es:** verifica la unidad de código más pequeña y aislable, una función o un método, sin depender de nada externo.
- 🔗 **¿Quién las escribe?** El propio equipo de desarrollo, muchas veces junto con el código (TDD).
- ⚡ **Características:** rápidas (milisegundos), 100% automatizadas, no tocan base de datos, red ni disco.

```
@Test
void deberiaAplicarDescuentoCorrecto() {
    double r = calc.aplicarDescuento(100, 10);
    assertEquals(90, r);
}
```

El secreto del aislamiento: mocks y stubs

Sin aislar las dependencias externas (base de datos, servicios, APIs), una "prueba unitaria" en realidad se convierte en una prueba de integración: lenta, frágil y dependiente del entorno.

Doble de prueba	Qué hace	Pregunta que responde
Stub	Devuelve datos fijos y predecibles, sin verificar cómo fue llamado.	"Cuando me pidan el usuario 42, devolveme siempre este objeto."
Mock	Simula el comportamiento y además verifica que se lo haya invocado correctamente.	"¿Se llamó una vez al método guardar() con estos parámetros?"

Cuando los módulos empiezan a hablar entre sí

Verifican que dos o más componentes, ya integrados, funcionen correctamente juntos: comunicación con bases de datos, colas de mensajes, APIs internas o microservicios.

01

Big Bang

Se integran todos los módulos a la vez y se prueban juntos.

02

Top-Down

Se prueba desde los módulos de alto nivel hacia abajo, usando stubs para lo que falta.

03

Bottom-Up

Se prueba desde los módulos más bajos hacia arriba, usando drivers.

04

Sandwich

Combina Top-Down y Bottom-Up, encontrándose en el medio.

Hoy, casi toda integración pasa por una API

¿Qué se prueba en una API?

- Código de estado HTTP (200, 400, 404, 500...)
- Estructura y datos del payload (JSON/XML)
- Headers, autenticación y autorización
- Tiempos de respuesta y manejo de errores

Herramientas que vamos a automatizar

-  Postman / Newman: exploración manual y CI
-  REST Assured: automatización en Java
-  Pytest + requests: automatización en Python

El software completo, de punta a punta

Se evalúa el sistema como un todo integrado, simulando el uso real (funcional y no funcional) en un entorno lo más parecido posible a producción.



Que lo viejo no se rompa con lo nuevo

No es un nivel nuevo: es una estrategia que se aplica sobre las pruebas unitarias, de integración y de sistema ya existentes.

¿Cuándo se corre? Después de cada cambio: una nueva funcionalidad, un fix, un refactor.

¿Por qué se automatiza? Para poder repetirla rápido y con frecuencia, en cada build o despliegue.

 **Se arregla un bug en Pagos**



 **Se corre la suite completa de Pagos + flujos críticos relacionados**

La validación final: ¿esto sirve de verdad?

- 👤 **¿Quién la ejecuta?** El usuario final o el cliente de negocio, no el equipo técnico.
- 📖 **¿Sobre qué se basa?** Escenarios reales de uso de negocio, casi nunca casos técnicos.
- 🏆 **¿Qué determina?** El "sí" o "no" final antes de salir a producción.

El rol de QA en UAT

Facilita las sesiones, prepara los escenarios de negocio y documenta cada hallazgo que reporta el usuario como un defecto formal.

La creatividad como herramienta de testing

No se basa en casos escritos de antemano: se diseñan y ejecutan pruebas **al mismo tiempo**, guiadas por la experiencia y la intuición del tester.

Se apoya en heurísticas: "*¿qué pasa si dejo este campo vacío?*", "*¿qué pasa si subo un archivo gigante?*"

- 🕒 Sesión de 60 minutos, misión clara
- 🗨️ Notas y hallazgos en tiempo real
- 💡 Session-Based Test Management (SBTM)

La pirámide de testing



Resumen de la clase

Nivel	¿Quién prueba?	Foco principal
Unitarias	Desarrollo	Una función o método, aislado con mocks/stubs
Integración	Dev / QA	Módulos y APIs comunicándose entre sí
Sistema / Regresión	QA	Flujo completo end-to-end, sin romper lo existente
UAT / Exploratorias	Usuario / QA	El software realmente sirve, más allá de lo esperado

► **Próxima clase:** automatización de APIs con REST Assured, Postman/Newman y Pytest.